# DENOISING DIFFUSION MODELS

**Milan Tóth**
milan.toth@uni-ulm.de
Ulm University
Ulm, Germany

**ABSTRACT**

Diffusion models generate images by reversing a process that gradually destroys them. Noise is added to an image over many steps until nothing of it remains, and a neural network is trained to undo one such step at a time; new images are then produced by applying that network repeatedly, starting from noise alone. This paper introduces the approach and follows its development. It begins with the general formulation proposed by Sohl-Dickstein et al., which leaves the form of the noise and the parameters of the reverse steps open, and shows how Denoising Diffusion Probabilistic Models arise from fixing these choices. Generating one image this way requires hundreds to thousands of network evaluations, and this is what much of the subsequent work addresses. Denoising diffusion implicit models show that the chain may be shortened by skipping over states, and a second line of work reopens the noise schedule and the reverse variance to similar effect, while latent diffusion models leave the number of steps untouched and instead make each one cheaper by operating on a compressed representation of the image. The paper closes with conditioning, which lets a model be steered by a class label or a text prompt, and with guidance, which controls how closely the generated image follows it.

***Keywords*** Diffusion Models · Generative Models · Image Generation · Denoising Diffusion Probabilistic Models · Latent Diffusion · Conditional Guidance

## 1 Introduction

Generative image modelling asks for samples from a distribution over images that is never observed directly, only through a finite collection of examples. The distribution is both extremely high-dimensional and sharply concentrated: almost every point in pixel space is noise, and the images of interest occupy a vanishing fraction of it.

Three properties are desirable at once: high sample quality, coverage of the modes of the data distribution, and fast sampling. Xiao et al. [26] observe that established model families typically achieve two of the three and compromise on the remaining one, and call this the generative learning trilemma. Generative adversarial networks [11] produce high-quality samples from a single forward pass but are reported to cover the data modes poorly, whereas variational autoencoders [14] and normalising flows [9, 15] cover them faithfully at a cost in sample quality [26].

Diffusion models deal with this trilemma differently, and the way they do so determines which property they give up. Their construction rests on an asymmetry [22]: destroying the structure of an image is easy, restoring it is not. What makes the problem tractable is that its difficulty depends on how much is destroyed at once. Reversing the process in one leap is hopeless, but

reversing a single sufficiently small increment of it is not, since the reversal of a small diffusion step has the same simple form as the step it undoes [10]. An image is therefore corrupted gradually, over many steps, until nothing of it remains but noise, and a neural network is trained to undo one such step at a time. Generation runs this chain backwards: noise is drawn at random and the network applied repeatedly until an image emerges [22, 12]. One hard inference problem is thereby exchanged for a long sequence of easy ones. Diffusion models are reported to achieve high sample quality together with good mode coverage [26]; what they give up is sampling speed, since the whole sequence must be traversed for every sample.

Sections 2 and 3 develop this construction, and Sections 4 and 5 are largely concerned with recovering the property it gives up. Section 6 then adds control over what is generated.

## 2 Foundations of Diffusion Models

Sohl-Dickstein et al. [22] introduce diffusion probabilistic models, a framework inspired by non-equilibrium statistical physics in which a tractable generative model is obtained by learning to reverse a gradual process that destroys structure in the data. This section presents the general framework they propose, including several design choices left open. Section 3 shows how Ho et al. [12] fix these choices to arrive at the concrete Denoising Diffusion Probabilistic Models (DDPM), making Sohl-Dickstein et al.'s work the direct theoretical foundation on which DDPM is built.

### 2.1 The Diffusion Probabilistic Model Framework

The framework defines a generative model through two Markov chains running in opposite directions. In the *forward trajectory*, a diffusion kernel $T_\pi$ is applied over and over to the data distribution $q(\mathbf{x}_0)$, degrading it in small increments until it has been driven into a simple target distribution $\pi(\mathbf{y})$ that can be treated analytically:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = T_\pi(\mathbf{x}_t \mid \mathbf{x}_{t-1}; \beta_t), \qquad q(\mathbf{x}_{0\cdots T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}), \tag{1}$$

where $\beta_t$ denotes the diffusion rate at step $t$. The kernel itself is left unspecified. Choosing $\pi(\mathbf{y})$ to be an isotropic Gaussian, for example, yields

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t;\, \mathbf{x}_{t-1}\sqrt{1-\beta_t},\; \beta_t \mathbf{I}\Big)\,, \tag{2}$$

whereas binary data can be handled by a binomial kernel instead. The diffusion rates $\beta_{1\cdots T}$ may either be held fixed or learned [22].

The *reverse trajectory* is the generative model. Sampling starts from $\pi(\mathbf{x}_T)$ and walks back along the chain one step at a time,

$$p(\mathbf{x}_T) = \pi(\mathbf{x}_T), \qquad p(\mathbf{x}_{0\cdots T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p(\mathbf{x}_{t-1} \mid \mathbf{x}_t). \tag{3}$$

The central insight enabling this construction is due to Feller [10] and forms the basis of the framework [22]: for sufficiently small diffusion rates $\beta_t$, the reversal $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ of a diffusion step belongs to the same family of distributions as the forward step it undoes. Learning the reverse process therefore reduces to estimating the parameters of a sequence of simple conditional distributions, rather than describing a complex distribution directly. In the Gaussian case, these parameters are the mean $f_\mu(\mathbf{x}_t, t)$ and the covariance $f_\Sigma(\mathbf{x}_t, t)$ of each reverse step. Because each individual step is only a small perturbation, this estimation problem is tractable even though the overall transformation between noise and data is highly complex [22].

Training maximises a lower bound $K$ on the model log-likelihood $L = \int d\mathbf{x}_0\, q(\mathbf{x}_0) \log p(\mathbf{x}_0)$. Jensen's inequality gives a bound that splits into KL divergences between the forward posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ and the learned reverse transitions, along with entropy terms available in closed form [22]. The full form of the bound is given in Appendix A. Density estimation is in this way recast as regression over the parameters of the reverse transitions.

### 2.2 Degrees of Freedom in the Original Formulation

The framework as presented is deliberately generic: it specifies the structure of the two Markov chains and the objective used to train them, but leaves the concrete realisation of several components open. Three choices in particular are left to the practitioner.

First, the diffusion kernel $T_\pi$ is not tied to a particular family of distributions. Continuous data may be diffused towards an isotropic Gaussian, while binary data can instead be handled by a binomial kernel, with the requirement being only that the kernel admits a tractable reversal [22].

Second, the diffusion rates $\beta_{1\dots T}$ need not be prescribed in advance. For Gaussian diffusion they can be optimised jointly with the model by gradient ascent on the bound $K$, rather than being fixed to a predetermined schedule [22].

Third, both parameters of each reverse transition are learned. In the Gaussian case the mean $f_\mu(\mathbf{x}_t, t)$ and the covariance $f_\Sigma(\mathbf{x}_t, t)$ are produced by a function approximator, which the framework does not constrain; multi-layer perceptrons and multi-scale convolutional networks are used in the original experiments [22].

Each of these degrees of freedom is resolved in a specific way by Ho et al. [12]. Section 3 shows that DDPM is obtained largely by restricting this framework rather than extending it.

## 3 Denoising Diffusion Probabilistic Models (DDPM)

Ho et al. [12] instantiate the framework of Section 2 for image synthesis. Rather than broadening it, they narrow it: the diffusion kernel is fixed to the Gaussian case, the rates $\beta_{1\dots T}$ follow a predetermined schedule instead of being learned, and the covariance of each reverse step is held constant so that only its mean remains to be estimated. These restrictions are what make the model practical to train at scale. Figure 1 shows the resulting pair of processes, starting from an image $\mathbf{x}_0$ drawn from the data distribution: a fixed forward chain $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ that adds Gaussian noise until the image is indistinguishable from $\mathcal{N}(\mathbf{0}, \mathbf{I})$, and a learned reverse chain $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ that recovers it step by step.

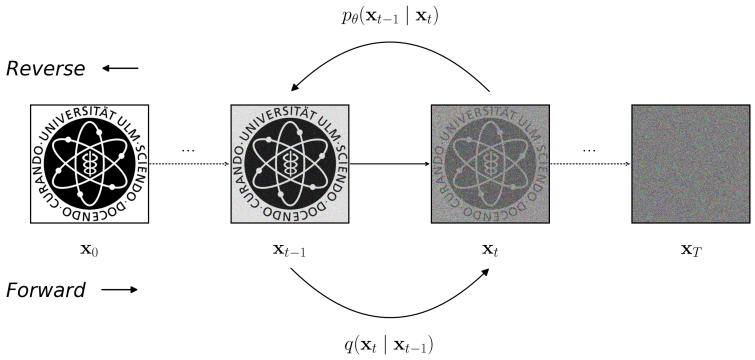



Figure 1: Forward and reverse diffusion process.

### 3.1 Forward Process

The forward process is the Gaussian kernel of Equation 2, applied to an image $\mathbf{x}_0$ over $T$ steps. The variances $\beta_{1\dots T}$ are fixed to constants rather than learned, so the forward process contains no trainable parameters at all: it is a fixed corruption procedure, and the entire modelling effort falls on the reverse process [12]. In their experiments the schedule is linear, increasing from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ over $T = 1000$ steps [12]; the values are small relative to data scaled to $[-1, 1]$, satisfying the condition of Section 2.1 while still driving $\mathbf{x}_T$ close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

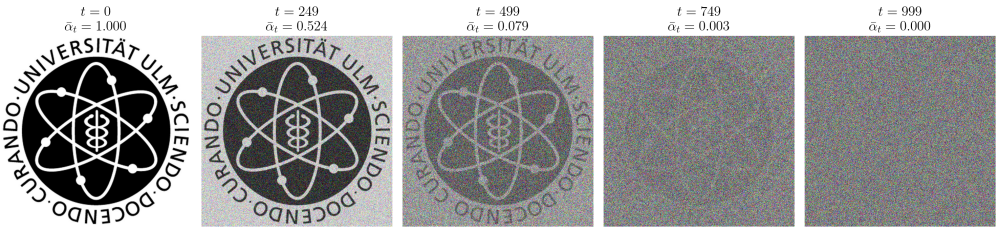



Figure 2: Forward diffusion process.

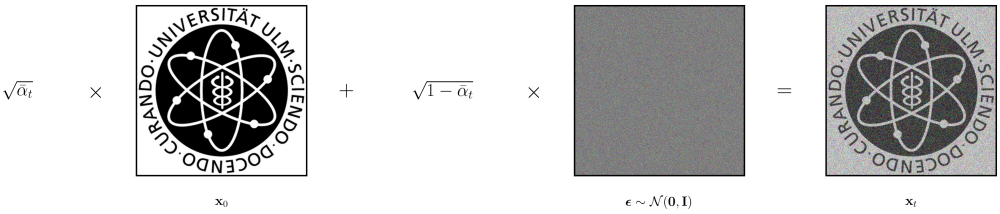



Figure 3: The closed form of Equation 4: $\mathbf{x}_t$ as a weighted sum of the image $\mathbf{x}_0$ and Gaussian noise $\boldsymbol{\epsilon}$, with weights determined by $\bar{\alpha}_t$.

The Gaussian kernel has a second consequence. Writing $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$, the composition of $t$ Gaussian steps is itself Gaussian, so $\mathbf{x}_t$ can be obtained from $\mathbf{x}_0$ in closed form [12],

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{4}$$

without iterating through the intermediate steps. Since $\bar{\alpha}_t$ decreases monotonically towards zero, $\mathbf{x}_t$ interpolates between the image at $t = 0$ and pure noise at $t = T$; Figure 2 shows this progression, and Figure 3 illustrates Equation 4 as a weighted sum of image and noise. The closed form is what makes training practical, since it allows a noisy sample at an arbitrary timestep to be drawn directly.

### 3.2 Reverse Process

The reverse process is again Gaussian, following Section 2.1, but with its covariance no longer learned:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2 \mathbf{I}\big)\,. \tag{5}$$

The covariance is set to untrained, time-dependent constants $\sigma_t^2 \mathbf{I}$ rather than being learned. Ho et al. [12] report two choices that perform comparably, $\sigma_t^2 = \beta_t$ and $\sigma_t^2 = \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$. Learning a diagonal covariance instead was found to destabilise training and worsen sample quality [12], which is the empirical justification for giving up this degree of freedom. Only the mean $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ therefore remains to be estimated.

The mean is not predicted directly. The variational bound of Section 2.1 decomposes as in Equation 17, with the terms for $t > 1$ comparing the learned reverse transition against the forward posterior, and because both distributions involved are Gaussian with fixed covariance, each such term reduces to a squared error between $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ and $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$, the mean of the forward posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$. This suggests a network predicting that posterior mean. Since the network receives only $\mathbf{x}_t$, the squared error is minimised not by $\tilde{\boldsymbol{\mu}}_t$ itself but by its expectation under $q(\mathbf{x}_0 \mid \mathbf{x}_t)$, so that the optimal mean is $\mathbb{E}_q[\mathbf{x}_{t-1} \mid \mathbf{x}_t]$ [2]. Substituting the closed form of Equation 4 eliminates $\mathbf{x}_0$ from $\tilde{\boldsymbol{\mu}}_t$ and leaves it a function of $\mathbf{x}_t$ and the noise $\boldsymbol{\epsilon}$ alone, so that predicting the mean and predicting the noise are equivalent tasks; Appendix B gives the derivation. Ho et al. [12] adopt the noise parameterisation, reporting that it performs comparably to predicting the mean under the variational bound, but considerably better when

**Algorithm 1** Training, adapted from [12]

1: **repeat**
2: $\quad \mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $\quad t \sim \mathrm{Uniform}(\{1, \dots, T\})$
4: $\quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: $\quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$
6: $\quad$ Take a gradient step on $\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t,\, t) \right\|^2$
7: **until** converged

**Algorithm 2** Sampling, adapted from [12]

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

combined with the simplified objective of Section 3.3. The network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ is trained to identify the noise present in $\mathbf{x}_t$, and a single reverse step becomes

$$\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right)}_{\boldsymbol{\mu}_\theta(\mathbf{x}_t,\, t)} + \sigma_t \mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{6}$$

Equation 6 draws a sample from $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ rather than returning its mean. The underbraced expression supplies that mean, and the variance $\sigma_t^2 \mathbf{I}$ is realised by sampling a fresh $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ at every step and scaling it by $\sigma_t$. Without this term the reverse process would be deterministic, mapping each $\mathbf{x}_T$ to a single fixed output. At the final step $t = 1$ the noise is omitted, so that the returned image is the mean rather than a perturbation of it [12]. The reparameterisation is also what gives the model its name: at each step the network acts as a denoiser, asked only to identify the noise contained in its input. Figure 4 shows the process applied to a trained model.

The network $\boldsymbol{\epsilon}_\theta$ is realised as a U-Net [21] with group normalisation in every block and self-attention inserted where the feature maps have a spatial size of $16 \times 16$ [12]. A single set of weights serves all timesteps, and $t$ is supplied to the network through a sinusoidal position embedding. Both the input $\mathbf{x}_t$ and the predicted noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ have the shape of the image itself: the network estimates one noise value per pixel and colour channel.

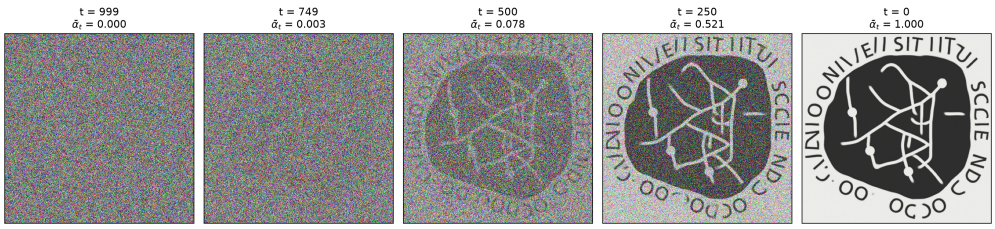



Figure 4: Reverse process of a model trained on a single image. A U-Net [21] following Ho et al. [12] was trained for 30 minutes on the Ulm University logo used as $\mathbf{x}_0$ throughout Section 3, then sampled from Gaussian noise. Details are given in Appendix C.

### 3.3 Training

Inserting the noise parameterisation of Section 3.2 into the variational bound of Section 2.1 leaves, for each timestep, a squared error between the true noise and the network's prediction,

$$L_{t-1} - C = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right\|^2\right], \tag{7}$$

where $\mathbf{x}_t$ is obtained from $\mathbf{x}_0$ and $\boldsymbol{\epsilon}$ through Equation 4; see Appendix B for the derivation [12].

Ho et al. [12] discard the leading coefficient and train instead on the unweighted objective

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right\|^2\right], \tag{8}$$

with $t$ drawn uniformly from $\{1, \dots, T\}$. This is no longer the variational bound but a reweighted version of it, and it is the point at which DDPM departs from the original formulation rather than merely restricting it. The reweighting is not neutral: under the linear schedule of Section 3.1 it reduces the influence of small $t$, where the input is barely corrupted and the denoising task is correspondingly easy, so that training concentrates on the harder, noisier timesteps [12]. Ho et al. [12] report that this improves sample quality while also being simpler to implement.

Algorithm 1 gives the resulting procedure. Each iteration draws an image and a timestep, constructs the corresponding noisy input in a single step through the closed form of Equation 4, and takes one gradient step on the squared error between the noise that was added and the noise the network predicts. No trajectory is ever simulated: the cost of an update does not depend on $T$, which is what makes training over $T = 1000$ steps practical. The network is also never trained on a fixed timestep, but on a fresh draw of $t$ at every iteration, so a single set of weights $\boldsymbol{\epsilon}_\theta$ learns to denoise across all noise levels.

### 3.4 Sampling

Generation reverses the process of Section 3.1. A sample is drawn from the prior, $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and Equation 6 is applied repeatedly until $\mathbf{x}_0$ is reached [12]; Algorithm 2 states the procedure. Line 3 is where the condition noted in Section 3.2 appears: fresh noise is added at every step except the last, where $\mathbf{z} = \mathbf{0}$ so that the returned image is the mean rather than a perturbation of it. Figure 4 shows the trajectory of a full sampling run, with intermediate states at four points along the chain.

The asymmetry with training is worth noting. Training touches a single, randomly chosen timestep per update, but sampling must traverse the entire chain, since $\mathbf{x}_{t-1}$ is required before $\mathbf{x}_{t-2}$ can be computed [12]. Generating one image therefore costs $T$ evaluations of $\boldsymbol{\epsilon}_\theta$, a thousand forward passes under the settings of Section 3.1. There is no closed-form shortcut here corresponding to Equation 4: that identity holds because the forward process is a fixed composition of Gaussians, whereas the reverse process passes through a learned network at every step. This sequential cost is the principal practical drawback of DDPM, and it motivates much of the subsequent work discussed in Section 4.

### 3.5 DDPM and the Generative Learning Trilemma

The three properties of Section 1 can now be taken up for the concrete model.

**Mode coverage** Diffusion models are placed on the covering side of the trilemma on empirical grounds: Xiao et al. [26] point to the high likelihoods reported for them as evidence of good mode coverage, and in their comparison, DDPM attains a higher recall than every conventional GAN for which recall is reported [26]. One explanation lies in the objective: the variational bound is a bound on the likelihood, and maximising likelihood minimises a divergence from the data to the model that favours placing probability wherever the data has mass, even at the cost of spreading it between modes [4]. The argument reaches $L_{\text{simple}}$ through the optimal mean it shares with the bound [2]. This leaves the opposite failure open, since each step targets a conditional expectation. Predictors of this kind are prone to the so-called regression to the mean: when several clean images are consistent with one corrupted input, the posterior expectation blends them instead of selecting one, an effect that Delbracio and Milanfar [7] largely avoid by restoring the image over many small steps rather than at once.

How the iterated sampler nevertheless commits to single modes has been described as a spontaneous symmetry breaking. While noise dominates, trajectories remain undecided between modes; at a critical time this state becomes unstable, and each trajectory commits to one mode [19]. Which mode is reached is decided by the noise around that time rather than by the starting point [3]. The commitment is not complete in practice: because the network learns only a smoothed version of a score that changes abruptly between modes, a small fraction of samples lands between neighbouring modes, more often when these are close, when training data are scarce, or when the chain is short [1]. Under the exact score of the training set, trajectories

would instead eventually collapse onto individual training points, which in practice a smoother learned score together with a sufficiently large dataset prevents [3]. Appendix D interprets these regimes in terms of the optimal reverse mean of Section 3.2.

**Sample quality** That diffusion models produce samples of high quality is likewise an empirical finding, which Xiao et al. [26] summarise as diffusion models beating GANs in image generation. Ho et al. [12] report sample quality competitive with GANs, and with an improved architecture and the classifier guidance of Section 6, Dhariwal and Nichol [8] surpass the state-of-the-art GANs of their time, at some cost in diversity. Explanations for this are partial and concern mainly the objective. $L_{\text{simple}}$ improves sample quality over the bound [12], and Choi et al. [6] show that different noise levels teach the network different things: at high noise it must learn recognisable content, whereas at low noise it can restore pixels from their clean neighbours and learns only imperceptible detail, so that shifting weight towards the former improves the samples further. Sample quality also depends on generating in many small steps. Generation in small steps largely avoids the blurring of a single-step prediction [7], and on a toy mixture DDPM loses sample quality when its chain is shortened, which Xiao et al. [26] trace to the Gaussian form of each reverse step being justified only for small steps. The property is thereby tied to the one diffusion models give up.

**Sampling speed** Sampling speed is what the construction gives up, and Section 3.4 has already shown the mechanism. What remains to be added is that the cost is not incidental: the same smallness of $\beta_t$ that keeps each reverse step simple enough to model is what makes so many of them necessary [26]. This argument, however, applies to the sampler of Section 3.4 as it stands: it assumes each reverse step is drawn from a Gaussian whose variance is fixed in advance. Section 4 relaxes these assumptions and recovers much of the lost speed.

# 4 Improvements of DDPM

This section presents two papers that build on the formulation of Section 3 without abandoning it. Both end up sampling in far fewer than $T$ steps, but arrive there differently. Song et al. [23] keep the trained model untouched and replace the sampling procedure, showing that the Markov assumption underlying the inference process can be relaxed. Nichol and Dhariwal [17] instead revisit the model itself, reopening the noise schedule and the reverse variance fixed in Section 3 and adapting the training objective accordingly.

## 4.1 Denoising Diffusion Implicit Models

Song et al. [23] quantify the cost identified in Section 3.4: a DDPM needs around 20 hours to generate 50k samples at a resolution of $32 \times 32$ on a single GPU, against under a minute for a GAN, and at $256 \times 256$ the time of the DDPM grows to almost 1000 hours.

Their observation is that the objective of Section 3.3 depends only on the marginals $q(\mathbf{x}_t \mid \mathbf{x}_0)$, never on the joint distribution $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$ over the whole trajectory. Many joints share the same marginals, and the Markov chain of Section 2.1 is only one of them. Among Markovian joints it is essentially the only one, since fixing the marginals of a Gaussian Markov chain leaves its transitions determined. Whatever freedom the objective permits must therefore lie among non-Markovian joints, and it is these that Song et al. [23] explore.

The family is constructed in the opposite direction to Section 3.1. Rather than defining the forward step and deriving the reverse conditional from it, Song et al. [23] specify

$$q_\sigma(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\cdot\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0}{\sqrt{1-\bar{\alpha}_t}},\ \sigma_t^2\mathbf{I}\right) \tag{9}$$

directly, indexed by a vector $\sigma \in \mathbb{R}^T_{\geq 0}$. The fraction is Equation 4 solved for $\boldsymbol{\epsilon}$: because both $\mathbf{x}_0$ and $\mathbf{x}_t$ are given, the noise that produced $\mathbf{x}_t$ is a known vector rather than a random one, and the mean carries it with weight $\sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}$.

Here $\sigma_t^2$ is a free parameter, and the mean is constructed around it so that the marginals of Equation 4 survive every choice. Squaring the two noise coefficients gives variances $1 - \bar{\alpha}_{t-1} - \sigma_t^2$ and $\sigma_t^2$, which sum to $1 - \bar{\alpha}_{t-1}$ irrespective of $\sigma_t$; the remaining term is $\sqrt{\bar{\alpha}_{t-1}}\,\mathbf{x}_0$. Evaluating Equation 4 at $t-1$ prescribes exactly this mean and variance, so $\sigma_t$ redistributes a fixed noise budget rather than altering it, and every member of the family reproduces the marginals a DDPM was trained on. Recovering the forward step by Bayes' rule, however, gives a conditional $q_\sigma(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)$ that retains a dependence on $\mathbf{x}_0$, which is what breaks the Markov property [23].

At sampling time $\mathbf{x}_0$ is unknown, so it is estimated by inverting Equation 4 with the network's prediction in place of the true noise [23]. This also removes the dependence that made the inference process non-Markovian: the estimate is computed from $\mathbf{x}_t$, so the generative process remains a Markov chain for every member of the family. Substituting the estimate into Equation 9 yields the generative step

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left( \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } \mathbf{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}_{\text{direction pointing to } \mathbf{x}_t} + \underbrace{\sigma_t \boldsymbol{\epsilon}_t}_{\text{random noise}}, \quad (10)$$

with $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ [23]. Each $\sigma$ induces its own variational objective, but Song et al. [23] show these differ from the objective of Section 3.3 only by per-timestep weights and an additive constant, and that the optimum is independent of those weights. That objective is therefore a valid surrogate for every member of the family, so a trained DDPM can be reused unchanged and only sampling is altered.

Two choices of $\sigma$ are of particular interest. Setting $\sigma_t^2 = \tilde{\beta}_t$ restores the Markov property and recovers DDPM exactly, so the sampler of Section 3.4 is one point in this family. Setting $\sigma_t = 0$ removes the noise term altogether: generation becomes deterministic and $\mathbf{x}_0$ a fixed function of $\mathbf{x}_T$. This is the case Song et al. [23] name the denoising diffusion implicit model (DDIM).

Sampling may also follow an increasing subsequence $\tau$ of $\{1, \ldots, T\}$ of length $S \ll T$, with each step spanning the gap between consecutive entries, since Equation 10 relates any two timesteps through their values of $\bar{\alpha}$. This applies to every member of the family, DDPM included; what distinguishes them is how well sample quality survives. For the deterministic case, Song et al. [23] find that 20 to 100 steps already suffice to match the samples of the full chain, a 10- to 50-fold speedup, whereas DDPM degrades sharply on short trajectories. Determinism additionally makes $\mathbf{x}_T$ an informative encoding: trajectories of different lengths from the same $\mathbf{x}_T$ share their high-level features, and interpolating between two latents produces semantically meaningful intermediates, neither of which holds for DDPM [23].

### 4.2 Improved Denoising Diffusion Probabilistic Models

Nichol and Dhariwal [17] address a different shortcoming: DDPM produces strong samples but log-likelihoods that lag behind other likelihood-based models. Lengthening the chain from $T = 1000$ to $T = 4000$ already improves log-likelihood, and three further changes reopen choices that Section 3 had fixed.

The first concerns the reverse variance. Nichol and Dhariwal [17] observe that the terms of the variational bound differ by orders of magnitude across timesteps, with the earliest few accounting for almost all of it, and that $\beta_t$ and $\tilde{\beta}_t$ diverge in precisely that region while nearly coinciding everywhere else. A fixed variance is thus a good approximation along most of the trajectory and a poor one exactly where the bound is most sensitive, which explains why the choice costs likelihood without visibly harming samples. Learning it directly remains difficult, since the two bounds leave only a narrow range to resolve [17], and this is the instability reported in Section 3.2. Nichol and Dhariwal [17] therefore have the network output a vector $\mathbf{v}$ and interpolate between the bounds in the log domain,

$$\Sigma_\theta(\mathbf{x}_t, t) = \exp\Big( \mathbf{v} \log \beta_t + (1 - \mathbf{v}) \log \tilde{\beta}_t \Big). \quad (11)$$

The second change follows from the first. Because $L_{\text{simple}}$ does not involve $\Sigma_\theta$, it supplies no gradient for $\mathbf{v}$, so the variational bound of Equation 17 is reintroduced as a correction,

$$L_{\text{hybrid}} = L_{\text{simple}} + \lambda L_{\text{vlb}}, \tag{12}$$

with $\lambda = 0.001$ and a stop-gradient on $\boldsymbol{\mu}_\theta$ in the second term, so that $L_{\text{vlb}}$ shapes only the variance [17].

The third change replaces the linear schedule of Section 3.1. Nichol and Dhariwal [17] find it well suited to high-resolution images but wasteful at $64 \times 64$ and below, where it drives $\bar{\alpha}_t$ towards zero faster than necessary and leaves the end of the trajectory almost pure noise. The cosine schedule is defined directly through $\bar{\alpha}_t$,

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2, \tag{13}$$

with a small offset $s = 0.008$. It falls off roughly linearly through the middle of the process and changes slowly near both ends, so information is destroyed more gradually.

Like Song et al. [23], Nichol and Dhariwal [17] sample on a subsequence of timesteps. Because Equation 11 expresses $\Sigma_\theta$ as a position between $\beta_t$ and $\tilde{\beta}_t$ rather than as an absolute value, recomputing those bounds for the shortened schedule rescales the learned variance automatically. Their $L_{\text{hybrid}}$ model retains near-optimal sample quality at $100$ steps despite being trained with $4000$, whereas models with fixed variances degrade markedly.

Their direct comparison finds DDIM ahead below $50$ sampling steps and behind at $50$ or more, so neither supersedes the other. What both establish is that the number of steps required is not intrinsic to diffusion models but follows from assumptions that can be relaxed: the Markov property of Section 2 in one case, the fixed variance and schedule of Section 3 in the other.

## 5 Latent Diffusion Models (LDM)

Both papers of Section 4 reduce the number of reverse steps, but neither changes what a single step costs. Every evaluation of $\boldsymbol{\epsilon}_\theta$ is a forward pass over a tensor the size of the image, and the same holds for every gradient computation during training. Rombach et al. [20] report that training the strongest pixel-space models of the time took between 150 and 1000 V100 days, and that generating 50k samples kept a single A100 busy for about five days [8]. Their stated aim is to lower the cost of training and of sampling alike, and thereby open the model class to wider use: training at this scale is possible only for a small part of the field, and a trained model remains costly to evaluate in time and memory.

To that end, Rombach et al. [20] ask how much of an image survives when it is described by only a limited number of bits. Using the rate–distortion measurement of Ho et al. [12] for a trained pixel-space diffusion model, they observe that the reconstruction error falls steeply as the first bits are added and then flattens: a small fraction of the bits carries nearly all of the perceptual content, while the rest encodes detail a viewer does not see. A likelihood-based model, which must account for the full distribution, spends capacity and compute on both [20]. The reweighted objective of Section 3.3 shifts how much weight the detail receives in the loss, but not what it costs, since $\boldsymbol{\epsilon}_\theta$ still processes a tensor of the full image size at every step. What does reduce the cost is discarding the detail before the model sees it. Rombach et al. [20] accordingly separate the perceptual compression that removes it from the semantic modelling that follows, and assign the two to different models.

The first of the two is an autoencoder. An encoder $\mathcal{E}$ maps an image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ to a latent $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{h \times w \times c}$, with a downsampling factor $f = H/h = W/w$ restricted to powers of two, so that the number of spatial positions falls by $f^2$. A decoder $\mathcal{D}$ maps latents back to images. The two are trained jointly to reconstruct their input, but are used apart from one another afterwards: diffusion training sees only latents produced by $\mathcal{E}$, while sampling starts from noise in latent space and applies $\mathcal{D}$ once at the end to turn the result into an image. The objective used to train the pair is designed to preserve detail that a pixel-wise error would average away; Rombach et al. [20] give the details.

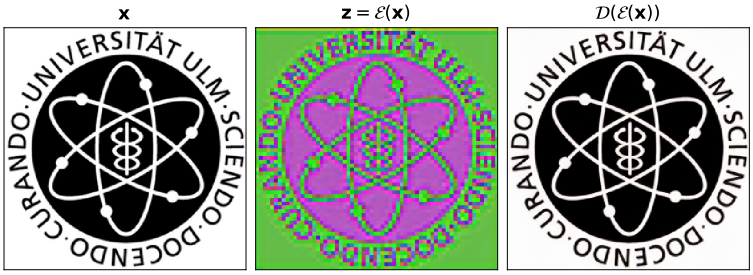



Figure 5: The image used as $\mathbf{x}_0$ throughout Section 3, its latent $\mathbf{z} = \mathcal{E}(\mathbf{x})$ under the autoencoder of Rombach et al. [20] with $f = 4$, and the reconstruction $\mathcal{D}(\mathcal{E}(\mathbf{x}))$. Encoding maps $256 \times 256 \times 3$ to $64 \times 64 \times 3$, a reduction by a factor of 16, while preserving the two-dimensional layout. The entries of $\mathbf{z}$ are unbounded, so its depiction as an RGB image follows the normalisation of Appendix C and is a display convention only.

The resulting latent is not merely smaller but retains the two-dimensional layout of the image, as the depiction in Figure 5 shows. Earlier two-stage models generated their latents autoregressively, which requires a fixed ordering of positions and therefore flattens the grid into a sequence, discarding its spatial structure and forcing high compression rates to keep sequence lengths feasible [20]. A diffusion model imposes no such ordering, and Rombach et al. [20] exploit this by keeping the convolutional U-Net of Section 3.2 as the second model: its inductive bias for spatial data applies to latents as it did to pixels, so mild compression suffices.

With $\mathbf{z}_0 = \mathcal{E}(\mathbf{x}_0)$ in place of $\mathbf{x}_0$, nothing in Section 3 changes. The objective is Equation 8 evaluated on latents, the forward process is still fixed so that $\mathbf{z}_t$ follows from Equation 4 in closed form, training follows Algorithm 1 unchanged, and sampling follows Algorithm 2 or one of the samplers of Section 4. Decoding a generated latent costs a single pass through $\mathcal{D}$ [20]. This is what makes a single step cheaper: $\boldsymbol{\epsilon}_\theta$ now processes a tensor with $f^2$ times fewer spatial positions, so the cost left untouched in Section 4 is exactly the one reduced here. The two reductions are independent and combine, which Rombach et al. [20] make use of by sampling with DDIM throughout their experiments.

The choice of $f$ is a two-sided trade-off. With small values, the autoencoder removes little and hands the bulk of the compression work back to the diffusion model, so training advances slowly, while large values discard information in the autoencoder and sample quality stops improving early in training. Rombach et al. [20] find $f = 4$ and $f = 8$ to be the best choices, as they combine efficient training with perceptually convincing samples.

The capacity a pixel-space model spends on imperceptible detail is, in a latent model, available for the structure that remains. Rombach et al. [20] isolate this by holding the budget fixed, training every model on a single GPU for the same number of steps and with the same number of parameters, and find the latent models to reach substantially better sample quality than the pixel-space model within that budget. Their latent models match or improve on the sample quality of the pixel-space diffusion models they compare against, while training on a fraction of the compute and with fewer parameters, and sampling at substantially higher throughput [20].

One limitation follows from the construction itself: the autoencoder bounds what can be recovered, so latent models are ill-suited to tasks in which results must be exact down to individual pixels [20]. Alongside latent diffusion, Rombach et al. [20] introduce a cross-attention mechanism that admits text, semantic maps and layouts as conditioning inputs, taken up in Section 6.

## 6 Conditioning and Guidance

Every model so far draws from $p(\mathbf{x})$, with no control over what appears in the sample. Conditioning on a variable $y$ requires little change to Section 3: the network receives $y$ as an additional input, becoming $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y)$, and training draws pairs $(\mathbf{x}_0, y)$ from the data while everything

else in Algorithm 1 stays as it is [13, 20]. What is left open is how $y$ reaches the network, which is architectural and settled at training time, and how closely the resulting sample follows it, which is settled at sampling time. This section takes the two in that order.

The first admits several answers depending on what $y$ looks like. A class label is a single symbol and needs almost nothing: Dhariwal and Nichol [8] project a class embedding through the same adaptive normalisation layers that already carry the timestep embedding into each residual block. Conditioning that is spatially aligned with the image, such as a semantic map or the low-resolution input of a super-resolution task, can be concatenated to the input channels of the U-Net [20]. Neither generalises to inputs of varying length. For those, Rombach et al. [20] introduce a cross-attention mechanism: an encoder $\tau_\theta$ maps $y$ to a sequence $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$, from which keys and values are formed, while queries come from an intermediate representation of the U-Net. Both networks are optimised jointly, and $\tau_\theta$ can be chosen to suit the modality at hand.

What such a model does not offer is a way to trade sample diversity for fidelity. Other generative models offer a lever for this, such as truncated sampling in a GAN [5], but the obvious analogues for diffusion models do not work. Rescaling the score estimate or shrinking the reverse-process noise both degrade samples rather than sharpening them [13, 8]. Guidance supplies the missing lever.

Dhariwal and Nichol [8] obtain it from a classifier. A model $p_\phi(y \mid \mathbf{x}_t)$ is trained on the same noising distribution as the diffusion model, so that it remains informative at every noise level, and its gradient with respect to the input is added to the predicted noise at each sampling step,

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - s\sqrt{1-\bar{\alpha}_t}\,\nabla_{\mathbf{x}_t} \log p_\phi(y \mid \mathbf{x}_t), \tag{14}$$

with a gradient scale $s$ [8]. Scaling the gradient is not cosmetic: since $s\nabla \log p_\phi(y \mid \mathbf{x}_t)$ is the gradient of a renormalised $p_\phi(y \mid \mathbf{x}_t)^s$, values $s > 1$ sharpen that distribution and concentrate sampling on its modes [8]. The effect is a trade-off, improving precision at the expense of recall [8]. Guidance can be applied to an unconditional model as well, though Dhariwal and Nichol [8] obtain their best results guiding a model that is already class-conditional.

Ho and Salimans [13] observe that the classifier is avoidable, and that avoiding it is desirable. Because it has to cope with noisy inputs, it cannot be taken off the shelf but must be trained specifically for this purpose. Moreover, moving a sample along the gradient of a classifier is precisely how adversarial examples are constructed, so it is unclear to what extent gains in metrics that are themselves computed by a classifier reflect genuinely better images. Their alternative trains a single network to serve both as a conditional and an unconditional model, replacing $y$ with a null token $\varnothing$ with probability $p_{\text{uncond}}$ during training. At sampling time the two predictions are combined,

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, y) = (1+w)\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, y) - w\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \varnothing), \tag{15}$$

where $w = 0$ leaves the conditional prediction untouched and larger $w$ extrapolates away from the unconditional one [13]. No classifier is involved and no separate model is trained, at the cost of two network evaluations per step instead of one. Ho and Salimans [13] find $p_{\text{uncond}} \in \{0.1, 0.2\}$ sufficient, with 0.5 performing worse across the entire trade-off curve.

Both methods trade diversity for fidelity in a similar manner. A small amount of guidance gives the best overall balance between the two, while stronger guidance yields samples that are individually more convincing but cover less of the distribution [13]. The gains are large enough that guidance is now routine: the best class-conditional result of Rombach et al. [20] in Section 5 is obtained with classifier-free guidance, which improves sample quality substantially over the same model sampled without it.

Text is one conditioning modality among these, requiring only an encoder that turns a prompt into a sequence and the cross-attention mechanism above. Nichol et al. [16] and Ramesh et al. [18] pursue this direction, and it is the subject of another paper in this seminar.

## 7 Summary and Outlook

The development traced here is largely one of narrowing and then selectively reopening. Sohl-Dickstein et al. [22] supply a framework general enough to leave the diffusion kernel, the noise schedule and both parameters of each reverse transition unspecified. Ho et al. [12] resolve all three, fixing the kernel to the Gaussian case, prescribing the schedule in advance and holding the reverse covariance constant so that only the mean remains to be learned. The resulting model is practical to train at scale not in spite of these restrictions but because of them. The one place where DDPM departs rather than restricts, the reweighted objective $L_{\text{simple}}$, is also reported to improve sample quality over the bound it replaces [12].

Generating a single image requires a full pass through the chain, one network evaluation per step, which is what makes sampling slow. This cost became the subject of the work that followed. Song et al. [23] show that the Markov assumption was never required by the objective. Sampling may then skip over states by following a subsequence of the timesteps, which the deterministic member of the resulting family tolerates far better than DDPM does, giving a 10- to 50-fold speedup with a trained DDPM reused unchanged. Nichol and Dhariwal [17] reopen the reverse variance and the noise schedule fixed in Section 3, adapting the training objective accordingly, and likewise sample on a subsequence of timesteps. Neither supersedes the other: their direct comparison finds DDIM ahead below 50 sampling steps and behind at 50 or more. Rombach et al. [20] address the cost of the individual step instead, moving the diffusion process into the latent space of an autoencoder, where the two reductions combine. Conditioning then supplies control over the sample, and guidance determines how closely that sample follows it [8, 20, 13].

Several limitations remain. Sampling is still sequential: shorter and cheaper per step than in Section 3, but a chain that must be traversed in full for every sample. The autoencoder of a latent model bounds what can be recovered, which matters wherever results must be exact down to individual pixels [20]. The position within the trilemma of Section 3.5 is therefore unchanged, and what the work discussed here has altered is the size of the price rather than which of the three properties is paid.

Ho et al. [12] note that their objective can be read as a form of denoising score matching, which places DDPM alongside the score-based generative models of Song and Ermon [24], trained directly on that objective. The formulation presented here is also discrete in time throughout. Song et al. [25] give a continuous-time view in which both lines of work appear as instances of a single framework.

## Acknowledgements

I thank Dr. Sebastian Gottwald for supervision and feedback throughout this project. I acknowledge support by the state of Baden-Württemberg through bwHPC.

## GenAI Usage Disclosure

I employed Claude for grammar and spelling correction, rephrasing and structural editing, as well as for discussing technical arguments, locating and checking sources, and drafting passages. I reviewed all content, verified it against the cited sources and revised it, and I take full responsibility for the paper.

## References

[1] Sumukh K Aithal et al. “Understanding hallucinations in diffusion models through mode interpolation”. In: *Advances in neural information processing systems* 37 (2024), pp. 134614–134644.

[2] Fan Bao et al. “Analytic-DPM: an analytic estimate of the optimal reverse variance in diffusion probabilistic models”. In: *arXiv preprint arXiv:2201.06503* (2022).

[3] Giulio Biroli et al. "Dynamical regimes of diffusion models". In: *Nature Communications* 15.1 (2024), p. 9957.

[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] Andrew Brock, Jeff Donahue, and Karen Simonyan. "Large Scale GAN Training for High Fidelity Natural Image Synthesis". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=B1xsqj09Fm.

[6] Jooyoung Choi et al. "Perception Prioritized Training of Diffusion Models". In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 11462–11471. DOI: 10.1109/CVPR52688.2022.01118.

[7] Mauricio Delbracio and Peyman Milanfar. "Inversion by direct iteration: An alternative to denoising diffusion for image restoration". In: *arXiv preprint arXiv:2303.11435* (2023).

[8] Prafulla Dhariwal and Alexander Nichol. "Diffusion Models Beat GANs on Image Synthesis". In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 8780–8794. URL: https://proceedings.neurips.cc/paper_files/paper/2021/file/49ad23d1ec9fa4bd8d77d02681df5cfa-Paper.pdf.

[9] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. "Density estimation using Real NVP". In: *International Conference on Learning Representations*. 2017. URL: https://openreview.net/forum?id=HkpbnH9lx.

[10] William Feller. "On the Theory of Stochastic Processes, with Particular Reference to Applications". In: *Proceedings of the [First] Berkeley Symposium on Mathematical Statistics and Probability*. University of California Press, 1949, pp. 403–432.

[11] Ian J. Goodfellow et al. "Generative Adversarial Nets". In: *Neural Information Processing Systems*. 2014. URL: https://api.semanticscholar.org/CorpusID:276883628.

[12] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5bcfec8584af0d967f1ab10179ca4b-Paper.pdf.

[13] Jonathan Ho and Tim Salimans. "Classifier-free diffusion guidance". In: *arXiv preprint arXiv:2207.12598* (2022).

[14] Diederik P. Kingma and Max Welling. "Auto-Encoding Variational Bayes". In: *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2014. URL: http://arxiv.org/abs/1312.6114.

[15] Durk P Kingma and Prafulla Dhariwal. "Glow: Generative Flow with Invertible 1x1 Convolutions". In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/file/d139db6a236200b21cc7f752979132d0-Paper.pdf.

[16] Alex Nichol et al. *GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models*. 2022. arXiv: 2112.10741 [cs.CV]. URL: https://arxiv.org/abs/2112.10741.

[17] Alexander Quinn Nichol and Prafulla Dhariwal. "Improved Denoising Diffusion Probabilistic Models". In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 8162–8171. URL: https://proceedings.mlr.press/v139/nichol21a.html.

[18] Aditya Ramesh et al. *Hierarchical Text-Conditional Image Generation with CLIP Latents*. 2022. arXiv: 2204.06125 [cs.CV]. URL: https://arxiv.org/abs/2204.06125.

[19] Gabriel Raya and Luca Ambrogioni. "Spontaneous symmetry breaking in generative diffusion models". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 66377–66389.

[20] Robin Rombach et al. "High-resolution image synthesis with latent diffusion models". In: *2022 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. ieee. 2022, pp. 10674–10685.

[21] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation". In: *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[22] Jascha Sohl-Dickstein et al. "Deep unsupervised learning using nonequilibrium thermodynamics". In: *International conference on machine learning*. pmlr. 2015, pp. 2256–2265.

[23] Jiaming Song, Chenlin Meng, and Stefano Ermon. "Denoising Diffusion Implicit Models". In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=St1giarCHLP.

[24] Yang Song and Stefano Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution". In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach et al. 2019, pp. 11895–11907. URL: https://proceedings.neurips.cc/paper/2019/hash/3001ef257407d5a371a96dcd947c7d93-Abstract.html.

[25] Yang Song et al. "Score-Based Generative Modeling through Stochastic Differential Equations". In: *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL: https://openreview.net/forum?id=PxTIG12RRHS.

[26] Zhisheng Xiao, Karsten Kreis, and Arash Vahdat. "Tackling the generative learning trilemma with denoising diffusion GANs". In: *arXiv preprint arXiv:2112.07804* (2021).

## A The Variational Bound

By Jensen's inequality, the model log-likelihood satisfies $L \geq K$, where for the diffusion trajectories considered here

$$K = -\sum_{t=2}^{T} \int d\mathbf{x}_0\, d\mathbf{x}_t\, q(\mathbf{x}_0, \mathbf{x}_t)\, D_{\mathrm{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) \\ + H_q(\mathbf{X}_T \mid \mathbf{X}_0) - H_q(\mathbf{X}_1 \mid \mathbf{X}_0) - H_p(\mathbf{X}_T), \quad (16)$$

with $H$ denoting entropy. Both the entropies and the KL divergences are available in closed form, since the forward posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is analytically tractable [22].

Ho et al. [12] write the same bound as a loss to be minimised, decomposed by timestep,

$$L_{\mathrm{vlb}} = \mathbb{E}_q \Big[ \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \,\|\, p(\mathbf{x}_T))}_{L_T} + \underbrace{-\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}_{L_0} \\ + \sum_{t>1} \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))}_{L_{t-1}} \Big], \quad (17)$$

where the name $L_{\mathrm{vlb}}$ follows Nichol and Dhariwal [17]. The terms $L_{t-1}$ are treated in Appendix B.

## B The Noise Parameterisation

The derivation in this appendix follows Ho et al. [12]. Conditioned on $\mathbf{x}_0$, the forward posterior in the KL terms of Appendix A is a tractable Gaussian, obtained by applying Bayes' rule to the forward kernel and Equation 4,

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1};\, \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\, \tilde{\beta}_t \mathbf{I}), \quad (18)$$

with mean

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t \tag{19}$$

and variance $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$. Since both Gaussians entering the divergence for step $t$ have fixed covariances, the divergence depends only on their means, and the contribution of step $t$ reduces to

$$L_{t-1} = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\right\|^2\right] + C, \tag{20}$$

where $C$ does not depend on $\theta$. Inverting Equation 4 expresses $\mathbf{x}_0$ through the noise that produced $\mathbf{x}_t$,

$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}\right). \tag{21}$$

Substituting this into Equation 19, using $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ and $\beta_t = 1-\alpha_t$, the dependence on $\mathbf{x}_0$ cancels:

$$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right). \tag{22}$$

Of the two quantities that determine this mean, the network receives $\mathbf{x}_t$ but not $\boldsymbol{\epsilon}$, so predicting the mean amounts to predicting the noise. Choosing $\boldsymbol{\mu}_\theta$ to mirror Equation 22 with $\boldsymbol{\epsilon}$ replaced by a network output $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ gives Equation 6, and the two means then differ only in their noise term,

$$\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = -\frac{1}{\sqrt{\alpha_t}}\cdot\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\left(\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right). \tag{23}$$

Substituting this into Equation 20 gives

$$L_{t-1} - C = \mathbb{E}_{\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right\|^2\right], \tag{24}$$

the objective of Section 3.3.

## C Demonstration Setup

The model in Figure 4 follows Section 3 but is deliberately small and trained on the single image used as $\mathbf{x}_0$ throughout that section. With one training example, the reverse process has only one target to converge to. It is therefore not a generative model in any meaningful sense, which is precisely what makes the trajectory legible as an illustration.

A U-Net [21] with four resolution levels ($64$, $128$, $256$, $256$ channels) and two residual blocks per level was trained for $3000$ steps with AdamW (learning rate $2 \times 10^{-4}$, batch size $4$), following Algorithm 1 with $T = 1000$ and the linear schedule of Section 3.1, but without the self-attention of Ho et al. [12]. Sampling followed Algorithm 2 over all $1000$ steps, with the five panels captured at evenly spaced points. Training and sampling ran on a single NVIDIA A100 GPU on bwUniCluster 3.0; all code uses PyTorch and the `diffusers` library.

The latent in Figure 5 was produced with a pretrained autoencoder released by Rombach et al. [20], namely the variant regularised by vector quantisation rather than a KL penalty, with $f = 4$ and three latent channels. The image was padded to a square on white, resized to $256 \times 256$ and scaled to $[-1, 1]$, giving $\mathbf{z} \in \mathbb{R}^{64\times64\times3}$. Following Rombach et al. [20], $\mathbf{z}$ is taken before the quantisation layer, which is treated as the first layer of the decoder, so the reconstruction passes through quantisation while the depicted latent does not. Encoding is a single forward pass on a CPU.

Since the entries of $\mathbf{z}$ are unbounded, each channel was clipped at its $0.5$-th and $99.5$-th percentiles and mapped linearly onto $[0, 1]$, and the three channels were assigned to red, green and blue in index order. Both choices serve display only and carry no meaning. The latent is enlarged without smoothing, so that each of its $64 \times 64$ positions remains visible as a distinct square.

# D An Interpretation of the Dynamical Regimes

Biroli et al. [3] distinguish three regimes of the backward process, separated by transitions they call speciation and collapse: trajectories first remain uncommitted, then commit to a class but not to a particular training image, and, under the exact score of the training set, finally collapse onto a training image. The regimes, their transition times and the collapse are results of Biroli et al. [3]. This appendix reads them in terms of the optimal reverse mean of Section 3.2, an interpretation which, to my knowledge, has not been stated in this form, although related formulations may exist.

For a network trained to optimality, $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ equals $\mathbb{E}_q[\mathbf{x}_{t-1} \mid \mathbf{x}_t]$, the expectation over all states at $t-1$ from which the forward process, started at the data, reaches $\mathbf{x}_t$. Since $\tilde{\boldsymbol{\mu}}_t$ is linear in $\mathbf{x}_0$, this equals $\tilde{\boldsymbol{\mu}}_t$ evaluated at $\mathbb{E}_q[\mathbf{x}_0 \mid \mathbf{x}_t]$ [2], so each step is governed by which data points remain plausible as the origin of $\mathbf{x}_t$. Under an exact score on a finite training set these are the training images; for a sufficiently large dataset this makes no difference in the first two regimes, where the noised empirical and population distributions essentially coincide [3].

At the beginning of sampling, almost any data point is a plausible origin, so $\mathbb{E}_q[\mathbf{x}_0 \mid \mathbf{x}_t]$ lies close to the data mean, and the step pulls only weakly towards it, since the weight of $\mathbf{x}_0$ in Equation 19 is small while $\bar{\alpha}_t$ is close to zero. The deterministic part of the step mainly contracts $\mathbf{x}_t$ towards the origin, the noise added in Equation 6 compensates this, and, apart from being kept centred, the trajectories perform the essentially Brownian motion that Biroli et al. [3] describe for the first regime, favouring no class.

As the noise decreases, the plausible origins narrow until they no longer cover all classes alike. A trajectory that fluctuations have moved towards one class then makes that class the more probable origin, and the next step pulls it further in the same direction. Biroli et al. [3] locate this speciation where the accumulated noise becomes comparable to the remaining spread of the data along its principal component, and show that two copies of a trajectory continued with independent noise well before this time end in the same class no more often than in different ones, so earlier fluctuations do not decide the outcome. Once committed, a trajectory stays within its class while the features of that class form [3]. Since the plausible origins are still spread over many images of the class, no single training image dominates the step, and the sample acquires the features of the class without reproducing a particular one. Biroli et al. [3] note that their analysis extends to subclasses, so the same mechanism may repeat at finer scales.

Under an exact score, the plausible origins eventually shrink to a single training image, which then attracts the trajectory. Biroli et al. [3] place this collapse where the regions around the individual noised training points no longer cover the noised data distribution, and show that it can be avoided at finite times only if the number of training points grows exponentially with the dimension. In practice, a learned score that is smoother than the exact one, together with a sufficiently large dataset, prevents it [3].